

EXPERIMENT – 6

Name: K ANUSRI

Roll No: 2520030432

Section: 8

Skill: Escape Sequence Handling, Process Creation, Program Execution and Error Handling

Title

Implementation of Escape Sequence Handling and Process Execution in a Shell.

Aim

To handle escaped spaces and special symbols while preserving characters, validate parsed arguments, create child processes, execute programs with arguments, handle execution errors, and manage the parent process.

Theory

Escape sequences allow special characters such as spaces and symbols to be preserved as part of an argument. A command parser processes escaped characters and creates an argument list. The `fork()` system call creates a child process, while `execvp()` replaces the child process with the requested program. The parent process waits for the child using `waitpid()`, and execution errors are reported when a command cannot be launched.

Algorithm

1. Start the program.
2. Read command input from the user.
3. Parse the input into arguments while handling escaped characters.
4. Preserve escaped spaces and special symbols in the corresponding arguments.
5. Display the parsed arguments.
6. Create a child process using `fork()`.
7. In the child process, execute the command using `execvp()`.
8. If execution fails, display an error message.
9. In the parent process, wait for the child process using `waitpid()`.
10. Display the child process completion status.
11. Repeat until the exit command is entered.
12. Stop the program.

Code

```

k_anusri@DESKTOP-TM208BAQ:~$ cat > main.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 300
#define MAX_ARGS 50
#define MAX_ARG_LEN 200

int parse_command(char *input, char *args[])
{
    static char tokens[MAX_ARGS][MAX_ARG_LEN];
    int count = 0;
    int i = 0;

    while (input[i] != '\0')
    {
        while (input[i] == ' ' || input[i] == '\t')
            i++;

        if (input[i] == '\0')
            break;

        int len = 0;

        while (input[i] != '\0' &&
            input[i] != ' ' &&
            input[i] != '\t')
        {
            if (input[i] == '\\')
            {
                i++;

                if (input[i] == '\0')
                    break;

                tokens[count][len++] = input[i];
                i++;
            }
            else
            {
                tokens[count][len++] = input[i];
                i++;
            }

            if (len >= MAX_ARG_LEN - 1)
                break;
        }

        tokens[count][len] = '\0';
        args[count] = tokens[count];

        count++;

        if (count >= MAX_ARGS - 1)
            break;
    }

    args[count] = NULL;
    return count;
}

```

```

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("=== OSSP Experiment 6 ===\n");
    printf("Escape Sequence Parser and Process Execution\n");
    printf("Type a command. Type exit to quit.\n\n");

    while (1)
    {
        printf("ossp> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
        {
            printf("Empty command.\n");
            continue;
        }

        int count = parse_command(input, args);

        printf("\nParsed Arguments:\n");

        for (int i = 0; i < count; i++)
            printf("Argument %d: [%s]\n", i + 1, args[i]);

        if (count == 0)
            continue;

        pid_t pid = fork();

        if (pid < 0)
        {
            perror("fork");
            continue;
        }

        if (pid == 0)
        {
            printf("\nChild Process PID: %d\n", getpid());
            printf("Executing: %s\n\n", args[0]);

            execvp(args[0], args);

            perror("Execution failed");
            exit(1);
        }
        else
        {
            int status;

            printf("\nParent Process PID: %d\n", getpid());

            waitpid(pid, &status, 0);

            if (WIFEXITED(status))
                printf("\nChild process completed.\n");
            else
                printf("\nChild process terminated abnormally.\n");
        }
    }

    printf("Exiting OSSP Shell...\n");

    return 0;
}
EOF

```

Execution Screenshots

1. Escape Sequence and Child Process Execution

```
gcc main.c -o ossp
./ossp
=== OSSP Experiment 6 ===
Escape Sequence Parser and Process Execution
Type a command. Type exit to quit.

ossp> echo hello

Parsed Arguments:
Argument 1: [echo]
Argument 2: [hello]

Parent Process PID: 914

Child Process PID: 915
Executing: echo

hello

Child process completed.
ossp> echo hello\ world

Parsed Arguments:
Argument 1: [echo]
Argument 2: [hello world]

Parent Process PID: 914

Child Process PID: 916
Executing: echo

hello world

Child process completed.
ossp> Argument 2: [hello world]

Parsed Arguments:
Argument 1: [Argument]
Argument 2: [2:]
Argument 3: [[hello]
Argument 4: [world]]

Parent Process PID: 914

Child Process PID: 917
Executing: Argument

Execution failed: No such file or directory

Child process completed.
ossp> echo hello\@world
```

2. Command Execution and Error Handling

```

Parsed Arguments:
Argument 1: [echo]
Argument 2: [hello@world]

Parent Process PID: 914

Child Process PID: 918
Executing: echo

hello@world

Child process completed.
ossp> Argument 2: [hello@world]

Parsed Arguments:
Argument 1: [Argument]
Argument 2: [2:]
Argument 3: [[hello@world]]

Parent Process PID: 914

Child Process PID: 919
Executing: Argument

Execution failed: No such file or directory

Child process completed.
ossp> ls -l

Parsed Arguments:
Argument 1: [ls]
Argument 2: [-l]

Parent Process PID: 914

Child Process PID: 920
Executing: ls

total 60
-rw-r--r-- 1 admindeeraaj admindeeraaj 25 Sep 18 04:42 Makefile
drwxr-xr-x 3 admindeeraaj admindeeraaj 4096 Aug 18 06:10 linux-system-calls
-rw-r--r-- 1 admindeeraaj admindeeraaj 2793 Sep 20 13:44 main.c
-rwxr-xr-x 1 admindeeraaj admindeeraaj 16632 Sep 20 13:44 ossp
-rwxr-xr-x 1 admindeeraaj admindeeraaj 16472 Jul 30 05:24 process
-rw-r--r-- 1 admindeeraaj admindeeraaj 861 Jul 30 05:23 process.c
-rw-r--r-- 1 admindeeraaj admindeeraaj 45 Aug 18 06:04 student.txt

Child process completed.
ossp> wrongcommand

Parsed Arguments:
Argument 1: [wrongcommand]

Parent Process PID: 914

Child Process PID: 921
Executing: wrongcommand

Execution failed: No such file or directory

Child process completed.
ossp> exit
Exiting OSSP Shell...

```

Result

The escape sequence parser and process execution program was executed successfully. Escaped spaces and special symbols were preserved in parsed arguments. Child processes were created and commands were executed using `execvp()`, while invalid commands produced an execution error and the parent process waited for child completion.